

DE LA PLUME ET DU TEXTE

Érasme, saisi par Holbein la plume à la main, corrige son texte mot à mot dans la marge : VS Code offre au code ce que l'écritoire offrait à l'humaniste, un espace pour écrire, corriger et reprendre, phrase après phrase.



FIG. VII. HANS HOLBEIN LE JEUNE, « ÉRASME ÉCRIVANT », 1523

Tutoriel complet de Visual Studio Code : installation, interface, extensions Python/Jupyter/débogueur, Git intégré et automatisation, en trois parcours de niveau.

SOMMAIRE

1. AUTOMATISER UNE CHAÎNE DE COMMANDES AVEC TASKS.JSON

2. ENVIRONNEMENTS DISTANTS : REMOTE-SSH ET DEV CONTAINERS

3. R, QUARTO ET L'EXÉCUTION MIXTE PYTHON/R

4. GIT AVANCÉ DANS VS CODE

5. QUALITÉ DE CODE : FORMATAGE À LA SAUVEGARDE ET TESTS INTÉGRÉS

6. DOCUMENTER ET PUBLIER UN PROJET REPRODUCTIBLE

Visual Studio Code (VS Code) est un éditeur de code gratuit, open-source et multiplateforme, développé par Microsoft. Il n'est pas un logiciel de géomatique en soi — contrairement à QGIS ou TerrSet — mais il est devenu l'environnement de travail standard pour écrire, déboguer et versionner les scripts Python (GeoPandas, PyQGIS, rasterio...) et R qui automatisent les traitements présentés dans les modules Le Compas et travaux pratiques. Ce module en fait un tutoriel complet, en trois pistes (choisis la tienne dans le filtre « Afficher ») : chacune se lit seule, du début à la fin.

1. AUTOMATISER UNE CHAÎNE DE COMMANDES AVEC TASKS.JSON

Un traitement complet enchaîne souvent plusieurs commandes distinctes : lancer un script de prétraitement, exécuter les tests, générer un export final. Retaper cette suite à chaque fois est aussi risqué que refaire une chaîne de clics QGIS à la main. Le fichier `.vscode/tasks.json` déclare une tâche (task) qui exécute cette suite en une seule action.

EXTRAIT DE `.VSCODE/TASKS.JSON` : UNE TÂCHE PERSONNALISÉE

```
{ "label": "Exporter les résultats", "type": "shell",  
  "command": "python export.py --format geojson", "group": {  
    "kind": "build", "isDefault": true } }
```

Créée via `Ctrl+Shift+P > Tasks: Configure Task`, puis déclenchée ensuite par `Ctrl+Shift+B` (tâche de build par défaut) ou `Ctrl+Shift+P > Tasks: Run Task` pour choisir parmi plusieurs tâches déclarées.

- Une tâche peut enchaîner plusieurs commandes (`dependsOn`) : par exemple lancer les tests avant l'export, et annuler l'export si les tests échouent
- Une tâche peut aussi surveiller un fichier de sortie et se relancer automatiquement (`problemMatcher`, `isBackground`) — utile pour un serveur de développement local ou un export qui s'actualise en continu

CHAMP DE TASKS.JSON	RÔLE
label	Nom affiché de la tâche dans la liste (Tasks: Run Task)
type	"shell" (commande système) ou "process" (exécutable lancé directement)
command	La commande exécutée, identique à ce qu'on taperait dans le terminal
group	Catégorie ("build", "test") ; isDefault: true la lie à Ctrl+Shift+B
dependsOn	Liste d'autres tâches à exécuter avant celle-ci
problemMatcher	Motif qui relie la sortie de la commande aux erreurs affichées dans l'éditeur

APPROFONDISSEMENT

2. ENVIRONNEMENTS DISTANTS : REMOTE-SSH ET DEV CONTAINERS

Un traitement lourd (classification sur une image satellite de plusieurs Go, entraînement d'un modèle) dépasse souvent les capacités d'un poste personnel et doit s'exécuter sur un serveur de calcul distant ou un cluster. VS Code peut piloter ce calcul distant sans jamais quitter l'interface locale.

REMOTE-SSH

- Extension Microsoft qui connecte VS Code à un serveur distant via SSH
- L'éditeur, le terminal et le débogueur s'exécutent réellement sur le serveur distant — seule l'interface reste locale
- Adapté à un cluster de calcul universitaire ou un serveur cloud où les données volumineuses résident déjà

DEV CONTAINERS

- Extension Microsoft qui ouvre le projet à l'intérieur d'un conteneur Docker défini par le projet lui-même

- Garantit un environnement strictement identique (versions de Python, de GDAL, de bibliothèques) pour tous les contributeurs, indépendamment de leur machine
- Le fichier `.devcontainer/devcontainer.json` décrit l'image Docker et les extensions à installer automatiquement à l'ouverture

ATTENTION

REMOTE-SSH N'EST PAS UN SIMPLE TRANSFERT DE FICHIERS

À la différence d'un client FTP qui copierait des fichiers, Remote-SSH exécute réellement l'éditeur et ses extensions côté serveur : ouvrir un notebook Jupyter en Remote-SSH lance le noyau Python sur le serveur distant, pas sur le poste local — c'est cette exécution distante, pas seulement l'affichage, qui permet de traiter des données trop volumineuses pour un poste personnel.

Sous Windows, une troisième option locale existe : WSL (Windows Subsystem for Linux), un vrai environnement Linux intégré au système. L'extension WSL de VS Code ouvre un projet directement à l'intérieur de cette distribution Linux, utile quand une bibliothèque géomatique (GDAL, certaines dépendances compilées) s'installe plus simplement sous Linux que nativement sous Windows.

SOLUTION	OÙ S'EXÉCUTE LE CODE	CAS D'USAGE TYPIQUE
Remote-SSH	Sur un serveur ou cluster distant	Calcul lourd, données déjà présentes sur le serveur
Dev Containers	Dans un conteneur Docker local ou distant	Environnement identique garanti pour toute l'équipe
WSL	Dans une distribution Linux locale (Windows uniquement)	Bibliothèques géomatiques plus simples à installer sous Linux

APPROFONDISSEMENT

3. R, QUARTO ET L'EXÉCUTION MIXTE PYTHON/R

Un projet de recherche géomatique combine parfois Python (traitement, GeoPandas) et R (statistique, cartographie avec `sf/tmap`, voir le module dédié). VS Code prend en

charge les deux dans le même éditeur, sans obliger à changer d'application.

- Extension R (REditorSupport) : coloration syntaxique, autocomplétion et exécution de scripts .R
- radian, une console R alternative plus riche (coloration, historique multi-lignes) que la console R par défaut, utilisable comme terminal intégré de VS Code à la place de Rterm
- Quarto (extension officielle) : successeur de R Markdown, exécute des blocs de code Python et R dans un même document .qmd, produisant un rapport ou une page web unique à partir des deux langages combinés
- Un même projet de recherche peut ainsi documenter un prétraitement GeoPandas en Python et une analyse spatiale (autocorrélation, régression) en R, dans deux blocs d'un même document Quarto, sans jongler entre deux éditeurs

EXTRAIT D'UN DOCUMENT QUARTO (.QMD) MÊLANT PYTHON ET R

```
```{python} gdf = gpd.read_file("parcelles.gpkg") ``` ```{r}  
library(sf) plot(st_read("parcelles.gpkg")) ```
```

Deux blocs de code, l'un Python l'autre R, dans le même fichier .qmd : Quarto exécute chacun avec son propre moteur puis assemble un document final unique (HTML, PDF) qui mélange les deux résultats.

#### VOIR AUSSI : APPROFONDIR : R, GGLOT2 ET LA CARTOGRAPHIE AVEC SF/TMAP

Le module De l'Instrument et du Nombre détaille la syntaxe R et son usage géomatique ; ce module montre comment l'exécuter concrètement aux côtés de Python dans VS Code.

#### APPROFONDISSEMENT

## 4. GIT AVANCÉ DANS VS CODE

- Rebase interactif (via l'extension GitLens, ou Ctrl+Shift+P > Git: Rebase) : réécrit l'historique local avant de le publier — fusionner plusieurs commits de correction mineurs en un seul, réordonner des commits — jamais sur une branche déjà partagée avec d'autres
- Gestion de branches directement depuis le panneau Source Control : créer, basculer (checkout), fusionner (merge) une branche sans quitter l'éditeur ni retourner en ligne de commande

- GitLens (extension tierce très utilisée) ajoute une annotation en marge de chaque ligne (blame) indiquant qui l'a modifiée en dernier et quand, et un explorateur d'historique par fichier

#### ATTENTION

### REBASE RÉÉCRIT L'HISTORIQUE : JAMAIS SUR UNE BRANCHE PARTAGÉE

Un rebase interactif modifie les identifiants (hash) des commits concernés. Sur une branche déjà poussée et utilisée par d'autres, cela force chacun à resynchroniser son propre historique — source fréquente de confusion. Réserver le rebase interactif à des commits encore purement locaux, jamais publiés.

#### EXTRAIT DE .GITIGNORE ADAPTÉ À UN PROJET GÉOMATIQUE

```
*.tif *.tiff /data/raw/ __pycache__/ .venv/
```

Exclut les rasters volumineux du suivi Git (un dépôt Git gère mal des fichiers binaires de plusieurs centaines de Mo), le dossier de données brutes, et l'environnement virtuel local — chacun régénérable ou téléchargeable séparément, donc inutile à versionner.

#### REMARQUE

### GIT LFS POUR LES FICHIERS VOLUMINEUX QU'IL FAUT TOUT DE MÊME VERSIONNER

Quand un raster ou un jeu de données volumineux doit malgré tout être suivi (par exemple une donnée de référence qui ne change presque jamais), Git LFS (Large File Storage) remplace le fichier dans l'historique Git par un simple pointeur léger, et stocke le contenu réel séparément — évitant qu'un dépôt Git classique ne gonfle démesurément à chaque nouvelle version d'un raster de plusieurs centaines de Mo.

#### APPROFONDISSEMENT

## 5. QUALITÉ DE CODE : FORMATAGE À LA SAUVEGARDE ET TESTS INTÉGRÉS

- `editor.formatOnSave`: `true` (dans `settings.json`, globalement ou par langage) reformate automatiquement chaque fichier à chaque Ctrl+S, garantissant un style cohérent sans discipline manuelle
- Test Explorer (panneau dédié, icône fiole dans la barre d'activité une fois un framework de test détecté) : détecte automatiquement les tests `pytest` ou `unittest` d'un projet, les affiche en arborescence, et permet de lancer ou déboguer un test individuel d'un clic
- Un test qui échoue peut être débogué exactement comme un script normal : point d'arrêt posé dans le test ou dans le code testé, puis lancement en mode débogage depuis le Test Explorer

### EXTRAIT DE SETTINGS.JSON : FORMATAGE ET LINTING AUTOMATIQUES

```
{ "editor.formatOnSave": true, "editor.codeActionsOnSave": {
 "source.fixAll": "explicit" } }
```

`formatOnSave` reformate le fichier à chaque enregistrement ; `codeActionsOnSave` avec `source.fixAll` applique aussi les corrections automatiques du linter (imports inutilisés retirés, etc.) au même moment, sans action manuelle.

## DEVOIR À RENDRE : DÉBOGAGE

TROUVER ET CORRIGER UN BUG DANS UN SCRIPT  
GEOPANDAS

Un script fourni charge un GeoPackage de parcelles, calcule leur surface en hectares, puis filtre celles de plus de 5 ha — mais le résultat retourne une liste vide alors que le jeu de données contient des parcelles bien plus grandes que 5 ha. Le bug vient d'un oubli de reprojection avant le calcul de surface (`gdf.area` calculé directement sur des coordonnées géographiques en degrés, jamais re-projetées en Lambert-93). Pose un point d'arrêt juste après la ligne de calcul de surface, lance le script en mode débogage (F5), inspecte la colonne `surface_ha` dans le panneau Variables pour repérer des valeurs aberrantes (de l'ordre de  $10^{-9}$ , cohérent avec un calcul en degrés carrés plutôt qu'en hectares), puis corrige en ajoutant la reprojection manquante (`gdf.to_crs(epsg=2154)`) avant le calcul.

- Un point d'arrêt est posé juste après le calcul de surface, avant le filtre
- Le panneau Variables est utilisé pour repérer la valeur aberrante de `surface_ha`, sans ajouter le moindre `print()`
- La correction ajoute la reprojection en CRS métrique avant le calcul de surface, comme le rappelle la mise en garde du module Le Compas

## APPROFONDISSEMENT

6. DOCUMENTER ET PUBLIER UN PROJET  
REPRODUCTIBLE

Un script qui fonctionne sur une seule machine, celle de son auteur, n'est pas encore un projet reproductible : n'importe qui doit pouvoir recréer exactement le même environnement de zéro, sur une autre machine, des mois plus tard.

- README.md à la racine du projet : objectif, instructions d'installation, exemple d'exécution — le premier fichier lu par quiconque découvre le dépôt, prévisualisable directement dans VS Code (Ctrl+Shift+V)

- requirements.txt (pip) ou environment.yml (conda) : liste exacte des bibliothèques et de leurs versions, générée automatiquement (pip freeze > requirements.txt) plutôt que retapée à la main
- Une fois ces fichiers présents, reproduire l'environnement complet tient en quelques commandes dans le terminal intégré : création d'un environnement virtuel, puis installation depuis requirements.txt ou environment.yml

FICHIER	RÔLE
README.md	Objectif du projet, installation, exemple d'exécution
requirements.txt	Liste des bibliothèques et versions exactes (pip)
environment.yml	Équivalent pour un environnement conda
.gitignore	Fichiers volontairement exclus du suivi Git (rasters, .venv, __pycache__)
.vscode/settings.json	Paramètres du projet (interpréteur, formatage) partagés avec l'équipe
.devcontainer/devcontainer.json	Description de l'environnement Docker reproductible (si utilisé)

#### REPRODUIRE UN ENVIRONNEMENT PYTHON DE ZÉRO

```
python -m venv .venv .venv\Scripts\activate pip install -r requirements.txt
```

Trois commandes dans le terminal intégré (activation légèrement différente sous macOS/Linux : source .venv/bin/activate) : créer l'environnement virtuel, l'activer, puis installer exactement les mêmes versions de bibliothèques que celles utilisées par l'auteur du projet.

## EXEMPLE

### UN PROJET REPRODUCTIBLE, DU DÉPÔT GIT À L'EXÉCUTION

Cloner un dépôt Git (`git clone`), ouvrir le dossier dans VS Code, laisser Dev Containers reconstruire l'environnement exact décrit par `devcontainer.json` (ou recréer un environnement virtuel depuis `requirements.txt`), sélectionner l'interpréteur correspondant, puis relancer le script : c'est cette chaîne complète, de zéro à un résultat identique, que documente un bon README.

- Bilan — à retenir : `tasks.json` déclenche une chaîne de commandes en une seule action (`Ctrl+Shift+B`) ; Remote-SSH exécute réellement l'éditeur sur un serveur distant, Dev Containers garantit un environnement Docker identique pour tous ; l'extension R et Quarto permettent d'exécuter Python et R dans un même document ; un `.gitignore` géomatique exclut les rasters volumineux, Git LFS les versionne malgré tout sous forme de pointeurs légers ; `editor.formatOnSave` et le Test Explorer automatisent la qualité de code ; `requirements.txt/environment.yml` et un README documentent un environnement intégralement reproductible.

#### VOIR AUSSI : VOIR AUSSI : PYQGIS EN PROFONDEUR

Le module De la Géométrie Pensive détaille la console Python de QGIS et l'automatisation par lot — l'autre moitié de la chaîne d'outils dont VS Code est l'éditeur.